

Unit 4.2

Assembly Language Programming – Part 3

CpE 3104 - Microprocessors

Shift Instructions

3. LOGICAL Instructions

Mnemonics: **SHL, SAL, SHR, SAR, ROL, RCL, ROR, RCR**

SHL reg/mem

SHL reg, 1
SHL reg, CL

SHL mem, 1
SHL mem, CL



SAL reg/mem

SAL reg, 1
SAL reg, CL

SAL mem, 1
SAL mem, CL



3. LOGICAL Instructions

Mnemonics: SHL, SAL, **SHR, SAR**, ROL, RCL, ROR, RCR

SHR reg/mem

SHR reg, 1
SHR reg, CL

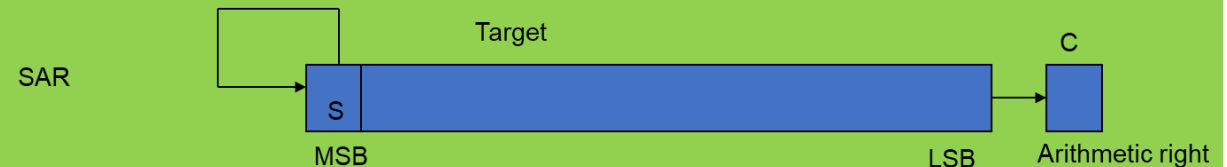
SHR mem, 1
SHR mem, CL



SAR reg/mem

SAR reg, 1
SAR reg, CL

SAR mem, 1
SAR mem, CL



Rotate Instructions

3. LOGICAL Instructions

Mnemonics: SHL, SAL, SHR, SAR, **ROL, RCL**, ROR, RCR

ROL reg/mem

ROL reg, 1
ROL reg, CL

ROL mem, 1
ROL mem, CL



RCL reg/mem

RCL reg, 1
RCL reg, CL

RCL mem, 1
RCL mem, CL



3. LOGICAL Instructions

Mnemonics: SHL, SAL, SHR, SAR, ROL, RCL, ROR, RCR

ROR reg/mem

ROR reg, 1
ROR reg, CL

ROR mem, 1
ROR mem, CL



RCR reg/mem

RCR reg, 1
RCR reg, CL

RCR mem, 1
RCR mem, CL



String Manipulation Instructions

4. STRING Manipulation Instructions

- ❑ **String** : Sequence of bytes or words
- ❑ 8086 instruction set includes instruction for string movement, comparison, scan, load and store.
- ❑ **REP** instruction prefix : used to repeat execution of string instructions
- ❑ String instructions end with **S** or **SB** or **SW**.
- ❑ S represents string, **SB** string byte and **SW** string word.
- ❑ Offset or effective address of the source operand is stored in **SI** register and that of the destination operand is stored in **DI** register.
- ❑ Depending on the status of **DF**, **SI** and **DI** registers are automatically updated.
- ❑ **DF** = 0 $\Rightarrow$ **SI** and **DI** are incremented by 1 for byte and 2 for word.
- ❑ **DF** = 1 $\Rightarrow$ **SI** and **DI** are decremented by 1 for byte and 2 for word.

4. STRING Manipulation Instructions

Mnemonics: **REP, MOVS, CMPS, SCAS, LODS, STOS**

REP

REPZ/ REPE

(Repeat CMPS or SCAS until ZF = 0)

While $CX \neq 0$ and $ZF = 1$, repeat execution of string instruction and
 $(CX) \leftarrow (CX) - 1$

REPNZ/ REPNE

(Repeat CMPS or SCAS until ZF = 1)

While $CX \neq 0$ and $ZF = 0$, repeat execution of string instruction and
 $(CX) \leftarrow (CX) - 1$

4. STRING Manipulation Instructions

Mnemonics: REP, **MOVS**, CMPS, SCAS, LODS, STOS

MOVS

MOVSB

$$MA = (DS) \times 16_{10} + (SI)$$

$$MA_E = (ES) \times 16_{10} + (DI)$$

$$(MA_E) \leftarrow (MA)$$

If DF = 0, then $(DI) \leftarrow (DI) + 1$; $(SI) \leftarrow (SI) + 1$
 If DF = 1, then $(DI) \leftarrow (DI) - 1$; $(SI) \leftarrow (SI) - 1$

MOVSW

$$MA = (DS) \times 16_{10} + (SI)$$

$$MA_E = (ES) \times 16_{10} + (DI)$$

$$(MA_E ; MA_E + 1) \leftarrow (MA ; MA + 1)$$

If DF = 0, then $(DI) \leftarrow (DI) + 2$; $(SI) \leftarrow (SI) + 2$
 If DF = 1, then $(DI) \leftarrow (DI) - 2$; $(SI) \leftarrow (SI) - 2$

4. STRING Manipulation Instructions

Mnemonics: REP, MOVSB, CMPS, SCAS, LODSB, STOS

Compare two string byte or string word

CMPS

CMPSB

$$MA = (DS) \times 16_{10} + (SI)$$

$$MA_E = (ES) \times 16_{10} + (DI)$$

Modify flags $\leftarrow (MA) - (MA_E)$

If $(MA) > (MA_E)$, then CF = 0; ZF = 0; SF = 0

If $(MA) < (MA_E)$, then CF = 1; ZF = 0; SF = 1

If $(MA) = (MA_E)$, then CF = 0; ZF = 1; SF = 0

CMPSW

For byte operation

If DF = 0, then $(DI) \leftarrow (DI) + 1$; $(SI) \leftarrow (SI) + 1$

If DF = 1, then $(DI) \leftarrow (DI) - 1$; $(SI) \leftarrow (SI) - 1$

For word operation

If DF = 0, then $(DI) \leftarrow (DI) + 2$; $(SI) \leftarrow (SI) + 2$

If DF = 1, then $(DI) \leftarrow (DI) - 2$; $(SI) \leftarrow (SI) - 2$

4. STRING Manipulation Instructions

Mnemonics: **REP, MOVS, CMPS, SCAS, LODS, STOS**

Scan (compare) a string byte or word with accumulator

SCAS

SCASB

$MA_E = (ES) \times 16_{10} + (DI)$
 Modify flags $\leftarrow (AL) - (MA_E)$

If $(AL) > (MA_E)$, then $CF = 0$; $ZF = 0$; $SF = 0$
 If $(AL) < (MA_E)$, then $CF = 1$; $ZF = 0$; $SF = 1$
 If $(AL) = (MA_E)$, then $CF = 0$; $ZF = 1$; $SF = 0$

If $DF = 0$, then $(DI) \leftarrow (DI) + 1$
 If $DF = 1$, then $(DI) \leftarrow (DI) - 1$

SCASW

$MA_E = (ES) \times 16_{10} + (DI)$
 Modify flags $\leftarrow (AX) - (MA_E)$

If $(AX) > (MA_E ; MA_E + 1)$, then $CF = 0$; $ZF = 0$; $SF = 0$
 If $(AX) < (MA_E ; MA_E + 1)$, then $CF = 1$; $ZF = 0$; $SF = 1$
 If $(AX) = (MA_E ; MA_E + 1)$, then $CF = 0$; $ZF = 1$; $SF = 0$

If $DF = 0$, then $(DI) \leftarrow (DI) + 2$
 If $DF = 1$, then $(DI) \leftarrow (DI) - 2$

4. STRING Manipulation Instructions

Mnemonics: REP, MOVS, CMPS, SCAS, LODS, STOS

Load string byte in to AL or string word in to AX

LODS	
LODSB	$MA = (DS) \times 16_{10} + (SI)$ $(AL) \leftarrow (MA)$ If $DF = 0$, then $(SI) \leftarrow (SI) + 1$ If $DF = 1$, then $(SI) \leftarrow (SI) - 1$
LODSW	$MA = (DS) \times 16_{10} + (SI)$ $(AX) \leftarrow (MA ; MA + 1)$ If $DF = 0$, then $(SI) \leftarrow (SI) + 2$ If $DF = 1$, then $(SI) \leftarrow (SI) - 2$

4. STRING Manipulation Instructions

Mnemonics: REP, MOVS, CMPS, SCAS, LODS, STOS

Store byte from AL or word from AX in to string

STOS	
STOSB	$MA_E = (ES) \times 16_{10} + (DI)$ $(MA_E) \leftarrow (AL)$ If DF = 0, then $(DI) \leftarrow (DI) + 1$ If DF = 1, then $(DI) \leftarrow (DI) - 1$
STOSW	$MA_E = (ES) \times 16_{10} + (DI)$ $(MA_E ; MA_E + 1) \leftarrow (AX)$ If DF = 0, then $(DI) \leftarrow (DI) + 2$ If DF = 1, then $(DI) \leftarrow (DI) - 2$

Process Control Instructions

5. Process Control Instructions

Mnemonics	Explanation
STC	Set CF $\leftarrow$ 1
CLC	Clear CF $\leftarrow$ 0
CMC	Complement carry CF $\leftarrow$ CF/
STD	Set direction flag DF $\leftarrow$ 1
CLD	Clear direction flag DF $\leftarrow$ 0
STI	Set interrupt enable flag IF $\leftarrow$ 1
CLI	Clear interrupt enable flag IF $\leftarrow$ 0
NOP	No operation
HLT	Halt after interrupt is set
WAIT	Wait for TEST pin active
ESC opcode mem/ reg	Used to pass instruction to a coprocessor which shares the address and data bus with the 8086
LOCK	Lock bus during next instruction

Control Transfer Instructions

6. Control Transfer Instructions

8086 conditional branch instructions affecting individual flags

Mnemonics	Explanation
JC disp8	Jump if CF = 1
JNC disp8	Jump if CF = 0
JP disp8	Jump if PF = 1
JNP disp8	Jump if PF = 0
JO disp8	Jump if OF = 1
JNO disp8	Jump if OF = 0
JS disp8	Jump if SF = 1
JNS disp8	Jump if SF = 0
JZ disp8	Jump if result is zero, i.e, Z = 1
JNZ disp8	Jump if result is not zero, i.e, Z = 1

6. Control Transfer Instructions

Transfer the control to a specific destination or target instruction
Do not affect flags

8086 Unconditional transfers

Mnemonics	Explanation
CALL reg/ mem/ disp16	Call subroutine
RET	Return from subroutine
JMP reg/ mem/ disp8/ disp16	Unconditional jump

6. Control Transfer Instructions

8086 signed conditional branch instructions

8086 unsigned conditional branch instructions

- Checks flags
- If conditions are true, the program control is transferred to the new memory location in the same segment by modifying the content of IP

6. Control Transfer Instructions

8086 signed conditional branch instructions

Name	Alternate name
JE disp8 Jump if equal	JZ disp8 Jump if result is 0
JNE disp8 Jump if not equal	JNZ disp8 Jump if not zero
JG disp8 Jump if greater	JNLE disp8 Jump if not less or equal
JGE disp8 Jump if greater than or equal	JNL disp8 Jump if not less
JL disp8 Jump if less than	JNGE disp8 Jump if not greater than or equal
JLE disp8 Jump if less than or equal	JNG disp8 Jump if not greater

8086 unsigned conditional branch instructions

Name	Alternate name
JE disp8 Jump if equal	JZ disp8 Jump if result is 0
JNE disp8 Jump if not equal	JNZ disp8 Jump if not zero
JA disp8 Jump if above	JNBE disp8 Jump if not below or equal
JAE disp8 Jump if above or equal	JNB disp8 Jump if not below
JB disp8 Jump if below	JNAE disp8 Jump if not above or equal
JBE disp8 Jump if below or equal	JNA disp8 Jump if not above